

 INTENSIVE MASTERCLASS

Python Bootcamp

*A Comprehensive 4-Module Hands-on Coding
Program*

🌿 INTENSIVE MASTERCLASS

Python Masterclass

🌿 What is python?

OR

🌿 WHat do you about python?

Masterclass Roadmap



Module 1

Syntax & Basics

Variables, data types, console I/O, and foundational Python rules.



Module 2

Decision Making

Conditional branching, comparison logic, and boolean flow control.



Module 3

Structures & Loops

Lists, Dicts, Tuples, Sets, and automated iteration techniques.



Module 4

Functions & Scope

Modular programming, return statements, and variable scoping rules.

MASTERCLASS 01

Syntax, Variables & Basics

Building a strong foundation in Python syntax and fundamental building blocks.

✓ Python Syntax & Indentation

✓ Variables & Dynamic Typing

✓ Basic Data Types (str, int, float, bool)

✓ User Input & Console Output

Core Syntax & Data Types

Understanding Variables & Types

Python is dynamically typed—no need to declare type explicitly!

Basic Syntax & Rules:

*Comments, indentation,
code structure, and
variable assignment.*

Data Types:

*Integers, floats, strings,
booleans, and type
conversion (int(), str()).*

Data Types:

*Integers, floats,
strings, booleans,
and type
conversion (int(),
str()).*

Module 1 Practice Set

Q1

Variable Swap Challenge

Write a Python script that takes two variables `a = 5` and `b = 10`, and swaps their values without manually hardcoding numbers.

Q2

User Age Calculator

Prompt the user to enter their birth year using `input()`, calculate their age, and output a formatted string showing their age.

Q3

Type Identifier

Given inputs `"100"`, `100`, and `99.9`, write commands to print the data type of each variable using `type()`.

MASTERCLASS 02

Decision Making in Python

Controlling code execution paths with conditional logic and Boolean evaluation.

✓ Conditional Branching (if, elif, else)

✓ Comparison & Relational Operators

✓ Logical Operators (and, or, not)

✓ Nested Conditions & Ternary Expressions

Control Flow Mechanics

If / Elif / Else Structure

Executes different blocks based on truth values.

Conditionals: *if, elif, else.*

Comparison & Logical Operators

Combining conditions and compare values

Comparison: *==, !=, >, <.*

Logical: *and, or, not.*

Module 2 Practice Set

Q1

Even or Odd Checker

Write a program that accepts an integer from the user and outputs whether the number is *Even* or *Odd* using the modulus operator %.

Q2

Maximum of Three Numbers

Prompt the user for 3 distinct numbers and determine which one is the largest using conditional statements without using the built-in `max()` function.

Q3

Leap Year Validator

Build a script that determines if a given year is a leap year (Divisible by 4, but not 100 unless also divisible by 400).

MASTERCLASS 03

Data Structures & Loops

Managing collections of data and automating repetitive execution effortlessly.

✓Python Lists, Tuples, Sets & Dictionaries

✓For Loops & Range Sequences

✓While Loops & Infinite Loop Prevention

✓Loop Control (break, continue, pass)

Collections & Iteration

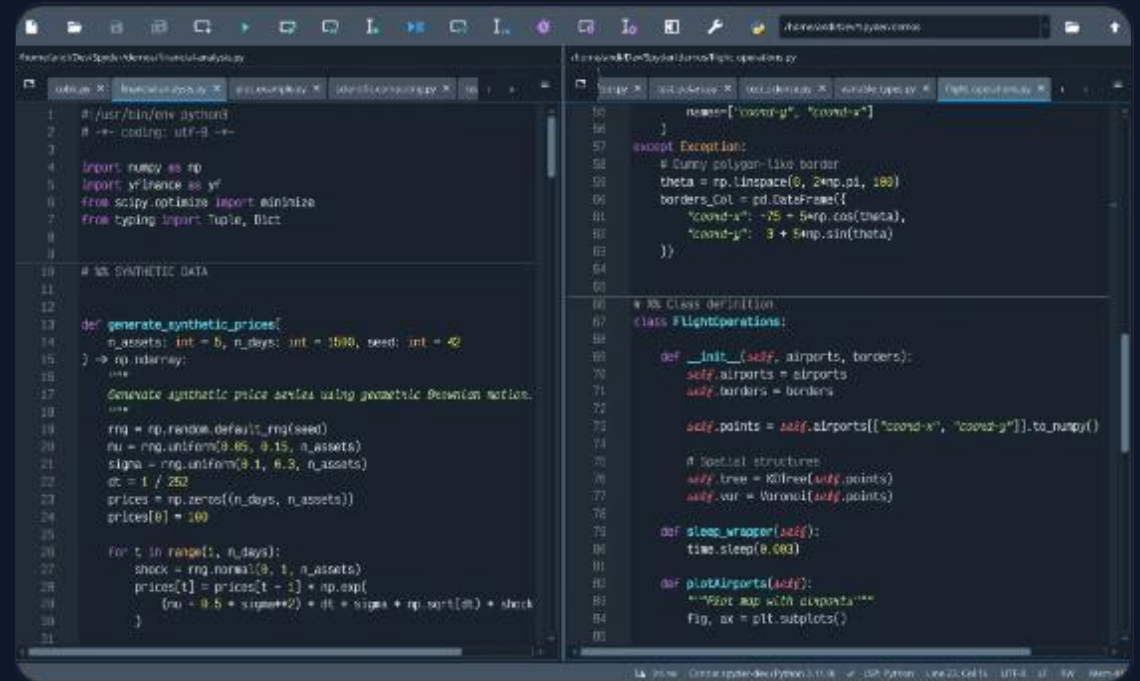
Data Collections at a Glance

List [1, 2]: Ordered, mutable, allows duplicates.

Tuple (1, 2): Ordered, immutable collection.

Dictionary {"a": 1}: Key-Value pairs for fast lookups.

Set {1, 2}: Unordered collection of unique items.



```
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 import numpy as np
5 import yfinance as yf
6 from scipy.optimize import minimize
7 from typing import Tuple, Dict
8
9
10 # %% SYNTHETIC DATA
11
12 def generate_synthetic_prices(
13     n_assets: int = 5, n_days: int = 1000, seed: int = 42
14 ) -> np.ndarray:
15     """
16     Generate synthetic price series using geometric Brownian motion.
17     """
18     rng = np.random.default_rng(seed)
19     mu = rng.uniform(0.05, 0.15, n_assets)
20     sigma = rng.uniform(0.1, 0.3, n_assets)
21     dt = 1 / 252
22     prices = np.zeros((n_days, n_assets))
23     prices[0] = 100
24
25     for t in range(1, n_days):
26         shock = rng.normal(0, 1, n_assets)
27         prices[t] = prices[t - 1] * np.exp(
28             (mu + 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * shock
29         )
30
31
32 class FlightOperations:
33     """
34     A class for managing flight operations.
35     """
36     def __init__(self, airports, borders):
37         self.airports = airports
38         self.borders = borders
39
40         self.points = self.airports[["coord-x", "coord-y"]].to_numpy()
41
42         # Spatial structures
43         self.tree = KDTree(self.points)
44         self.voronoi = Voronoi(self.points)
45
46     def sleep_wrapper(self):
47         time.sleep(0.001)
48
49     def plotAirports(self):
50         """Plot map with airports"""
51         fig, ax = plt.subplots()
```

Module 3 Practice Set

Q1

Sum & Average of List

Create a list of 5 numbers. Write a for loop that calculates the total sum and average of all numbers without using `sum()`.

Q2

Dictionary Word Counter

Given a sentence string, iterate through words and count word frequencies using a Python dictionary.

Q3

Number Guessing Game

Implement a while loop that continues to prompt the user to guess a secret number until they guess correctly, utilizing `break`.

MASTERCLASS 04

Functions & Scope

Writing modular, reusable, and structured code using modern function definitions.

✓ Defining Functions (def keyword)

✓ Parameters, Default Args & *args/**kwargs

✓ Return Values & Multiple Outputs

✓ Local vs Global Variable Scope

Modular Code & Scope Rules

Function Definition Syntax

Encapsulate logic for reusability.

Scope Rules: The LEGB Rule

Python looks up variable names in a strict order using the LEGB rule:

- [L] Local -> Inside the current function / method*
- [E] Enclosing -> Inside outer functions (nested functions)*
- [G] Global -> At the top level of the .py script / module*
- [B] Built-in -> Python's pre-loaded tools (e.g., len, print, range)*

Example

```
# helpers.py

APP_VERSION = "1.0.0"

def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

Module 4 Practice Set

Q1

Palindrome Function

Write a function `is_palindrome(text)` that returns `True` if a string reads the same forwards and backwards, ignoring spaces and casing.

Q2

Flexible Multiplier (*args)

Create a function `multiply_all(*numbers)` that takes any quantity of numeric arguments and returns their total product.

Q3

Scope Modification Experiment

Demonstrate the difference between modifying a global variable directly vs using the `global` keyword inside a function body.

 CONGRATULATIONS!

Bootcamp Complete

You've mastered the 4 core pillars of Python! Questions or ready for advanced projects?

 Happy Coding!

 Q&A Session